Domain-Driven Design Analysis

EasyParkPlus Parking Management System

The conversation with Michael can be found here.

1. BUSINESS CONTEXT

Company Profile

- **Current Operations:** 10 multi-story garages (Boston, NY, Philadelphia)
- **2-Year Plan:** 14-16 facilities in same metro regions
- **Business Model:** Hourly + time-banded pricing with membership tiers
- **Key Differentiator:** Integrated EV charging with multiple connector types

Target Markets

- Individual consumers
- Corporate accounts (with departmental billing)
- Fleet management
- Guest/one-time users

2. SUBDOMAIN ANALYSIS

2.1 Core Subdomains

A. Parking Space Management (Core)

Why Core: Revenue driver, customer-facing, requires optimization

Business Rules:

- Reservations: Up to 30 days advance, by slot type (not specific slot)
- Auto-assignment: Entrance-nearest, then load-balance across levels
- Real-time occupancy tracking
- Multi-facility operations (10+ garages)
- Access via LPR + Mobile QR codes
- Integration with existing gate hardware required

Pricing Model:

```
Hourly Rates:
- Regular: $3/hour
- EV: $4/hour
- Handicap: $1.50/hour
- Compact: $2/hour

Peak Hours (Weekdays 07:00-10:00, 16:00-19:00):
```

```
- Higher rates apply
- Some EV slots reserved during peak

Caps:
- Daily: $25
- Weekly: $120
- Monthly: $300

Special Rules:
- 15-minute exit grace period after payment
- Max duration: 72 hours (varies by facility)
- Overnight allowed at some sites (different rate)
- Lost ticket: Lookup by plate + max daily rate or flat fee
```

B. EV Charging Management (Core)

Why Core: Market differentiator, growing revenue stream

Business Rules:

- EV slots prefer charging use (parking-only at higher rate)
- Charge completion: 15-min grace, then overstay fee
- Per facility: 8-16 chargers (varies)
- Start: Plug + authorize via app or RFID
- User can set target SOC (State of Charge) via app
- Remote stop supported

Infrastructure:

```
Connector Types:
- Type 2
- CCS (Combined Charging System)
- CHAdeMO

Power Levels:
- Level 2 (7-19 kW): $0.35/kWh
- DC Fast (50-150 kW): $0.50/kWh
- Ultra-Fast (150-350 kW): $0.50/kWh

Billing:
- Per kWh primary pricing
- Optional per-minute idle/overstay fees
- Separate from parking billing

Monitoring:
- Real-time station health monitoring
- Track: kWh, duration, peak power, SOC start/end, errors
- Predictive maintenance: Nice to have
```

2.2 Supporting Subdomains

C. Customer Management

Account Types:

- Individual consumers
- Corporate accounts (multiple vehicles/users, departmental billing, reports)
- Fleet management
- Guest users (pay-by-plate or guest checkout)

Membership Benefits:

- Pricing discounts (tier-based)
- Priority access during peak
- Reserved slots capability

D. Billing Context

Rules:

- Parking and charging billed separately
- Failed payment: Invoice + hold exit for members
- Payment methods: Cards, Apple/Google Pay, PayPal, pre-paid, corporate billing, cash at kiosks (optional)
- Data retention: 5 years
- Compliance: CCPA (US operations)

E. Facility Management

Special Slot Types:

- ADA handicap
- Family/pregnant
- Motorcycle areas
- Oversized vehicle zones
- Carpool stalls
- Present at one or more facilities

2.3 Generic Subdomains

F. Notification Service

- Email, SMS, push notifications
- Grace period warnings
- Charge completion alerts
- Payment notifications

G. Integration Hub

Required Integrations:

- Google Maps (availability overlay)
- Apple Maps (availability overlay)
- Existing payment gateway
- Existing CRM
- Existing gate hardware (LPR + barriers)

H. Reporting & Analytics

Required Reports:

- Daily occupancy
- Revenue by facility
- Charging statistics
- Energy consumption
- Peak hour analysis

Access By:

- Facility managers
- Regional operations
- Finance team
- Executive management

3. BOUNDED CONTEXTS

For all key terms in this section: *See `ubiquitous_language.pdf` for definitions.*

BC1: Parking Operations Context (Core)

Responsibilities:

- Slot reservation (30 days advance)
- Real-time occupancy tracking across 10+ facilities
- Auto-assignment algorithm (entrance-nearest + load balancing)
- Multi-slot type management (Regular, EV, Handicap, Compact, Oversized, Motorcycle, Carpool)
- Access control integration (LPR + Mobile QR)
- Grace period tracking (15 minutes)
- Maximum duration enforcement (72 hours)

Core concepts: Reservation, Auto-assignment, Grace Period, Peak Hours, Max Duration, Overstay, LPR, Occupancy, Slot Type

Specialized terms: Time-banded Rate, Lost Ticket Protocol, Load Balancing

BC2: EV Charging Context (Core)

Responsibilities:

- Multi-connector charging (Type 2, CCS, CHAdeMO)

- Multi-power level support (Level 2, DC Fast, Ultra-Fast)
- Target SOC (State of Charge) management
- Remote session control via app
- Real-time station health monitoring
- Session data tracking (kWh, duration, peak power, SOC, errors)
- Overstay management (15-min grace after full charge)

Core concepts: SOC (State of Charge), Target SOC, Charging Session, kWh Rate, Power Level, Connector Type

Specialized terms: Idle Fee, Station Health, Predictive Maintenance

BC3: Customer Management Context (Supporting)

Responsibilities:

- Individual, corporate, fleet, and guest account management
- Multi-vehicle per account support
- Departmental billing for corporate accounts
- Usage report generation per user/vehicle
- Payment method management

Core concepts: Corporate Account, Fleet Account, Guest User, Departmental Billing, Membership Tier

Specialized terms: Pay-by-Plate, Usage Report

BC4: Billing Context (Supporting)

Responsibilities:

- Time-banded parking fee calculation
- Peak hour rate adjustment
- Separate EV charging fee calculation (per kWh + idle fees)
- Daily/weekly/monthly cap enforcement
- Multi-payment method processing
- Failed payment recovery (invoice + exit hold)
- 5-year data retention
- CCPA compliance

Core concepts: Time-banded Rate, Peak Rate, Rate Cap, Separate Billing, Exit Hold, Invoice

Specialized terms: CCPA Compliance, 5-Year Retention

BC5: Facility Management Context (Supporting)

Responsibilities:

- Multi-facility configuration (10+ garages, growing to 14-16)
- Special slot designation (ADA, family, motorcycle, oversized, carpool)
- Capacity management per facility
- Gate hardware integration (LPR + barriers)
- Maintenance scheduling

Core concepts: Multi-Facility, Special Slot, Gate Hardware, Facility Capacity

Specialized terms: Regional Operations, Maintenance Schedule

4. DOMAIN MODELS

> **Note:** Domain models define the core objects, their attributes, and behaviors within each bounded context. For complete term definitions and examples, please see `ubiquitous_language.pdf`.

4.1 Parking Operations Context

Aggregate: ParkingFacility

```
class ParkingFacility (AggregateRoot):
    # Identity
    - facilityId: FacilityId
    - facilityCode: String  # e.g., "BOS-01", "NYC-02"
    - name: String
    - address: Address
    - city: City  # Boston, NY, Philly

    # Structure
    - levels: List<ParkingLevel>
    - totalRegularSlots: int
    - totalEVSlots: int
    - totalHandicapSlots: int
    - totalCompactSlots: int
    - totalOversizedSlots: int
    - totalMotorcycleSlots: int
    - totalCarpoolSlots: int

    # Operations
    - operatingHours: OperatingHours
    - allowsOvernight: boolean
    - overnightRate: Money?
    - maxDurationHours: int  # Default 72, varies by facility
    - status: FacilityStatus

    # Access Control
    - hasLPR: boolean
    - hasMobileQR: boolean
    - gateHardwareModel: String  # For integration

    Methods:
    + reserveSlot(slotType: SlotType, startTime: DateTime, duration: Duration):
Reservation
    + autoAssignSlot(vehicleType: VehicleType, slotType: SlotType): ParkingSlot?
    + getTotalCapacity(): int
    + getAvailableSlots(slotType: SlotType): int
    + getOccupancyRate(): Percentage
    + isPeakHour(currentTime: DateTime): boolean
```

```
    + findNearestAvailableSlot(slotType: SlotType): ParkingSlot?
    + getLoadBalancedLevel(): ParkingLevel
```

Entity: Reservation

```
class Reservation (Entity):
    - reservationId: ReservationId
    - facility: ParkingFacility
    - customer: CustomerId
    - vehicle: VehicleId
    - slotType: SlotType
    - reservationTime: DateTime
    - startTime: DateTime
    - duration: Duration
    - status: ReservationStatus  # PENDING, CONFIRMED, ACTIVE, COMPLETED,
CANCELLED, NO_SHOW
    - assignedSlot: ParkingSlot?  # Assigned upon arrival

    Methods:
    + activate(): Result  # When vehicle arrives
    + cancel(): Result
    + markNoShow(): Result
    + canModify(): boolean  # Within cancellation policy
```

Entity: ParkingSlot

```
abstract class ParkingSlot (Entity):
    - slotId: SlotId
    - slotNumber: String
    - level: ParkingLevel
    - slotType: SlotType  # REGULAR, EV, HANDICAP, COMPACT, OVERSIZED, MOTORCYCLE,
CARPOOL
    - currentVehicle: Vehicle?
    - currentReservation: Reservation?
    - status: SlotStatus  # AVAILABLE, OCCUPIED, RESERVED, MAINTENANCE
    - entryTime: DateTime?
    - exitDeadline: DateTime?  # Entry + maxDuration

    Methods:
    + park(vehicle: Vehicle): Result
    + release(): ParkingSession
    + reserve(reservation: Reservation): Result
    + isAvailable(): boolean
    + isInGracePeriod(currentTime: DateTime): boolean  # 15 minutes
    + hasExceededMaxDuration(currentTime: DateTime): boolean
    + calculateParkingFee(exitTime: DateTime): Money
```

Value Object: ParkingSession

```
class ParkingSession (ValueObject):
    - sessionId: SessionId
    - vehicle: Vehicle
    - slot: ParkingSlot
    - entryTime: DateTime
    - exitTime: DateTime
    - duration: Duration
    - wasPeakHours: boolean
    - parkingFee: Money

    Methods:
    + getDuration(): Duration
    + crossedPeakHours(): boolean
```

Enumerations:

```
enum SlotType:
    REGULAR
    EV
    HANDICAP
    COMPACT
    OVERSIZED
    MOTORCYCLE
    CARPOOL

enum ReservationStatus:
    PENDING      # Reservation created, not yet confirmed
    CONFIRMED    # Reservation confirmed, awaiting arrival
    ACTIVE       # Vehicle has arrived, slot in use
    COMPLETED    # Reservation fulfilled
    CANCELLED    # Customer cancelled
    NO_SHOW      # Customer didn't arrive

enum City:
    BOSTON
    NEW_YORK
    PHILADELPHIA
```

4.2 EV Charging Context

Aggregate: ChargingStation

```
class ChargingStation (AggregateRoot):
    # Identity
    - stationId: StationId
```

```
    - stationCode: String  # e.g., "BOS-01-CS-05"
    - facility: FacilityId
    - location: Location

    # Hardware
    - connectorType: ConnectorType  # TYPE2, CCS, CHAdeMO
    - powerLevel: PowerLevel  # LEVEL2, DC_FAST, ULTRA_FAST
    - maxPowerOutput: PowerOutput  # kW

    # Status
    - status: StationStatus  # AVAILABLE, CHARGING, OFFLINE, MAINTENANCE
    - healthStatus: HealthStatus  # HEALTHY, DEGRADED, FAULTY
    - lastMaintenanceDate: DateTime
    - currentSession: ChargingSession?

    # Monitoring
    - totalEnergyDispensed: Energy  # Lifetime kWh
    - totalSessions: int
    - errorCount: int
    - lastErrorTime: DateTime?

    Methods:
    + startCharging(vehicle: ElectricVehicle, targetSOC: Percentage?, authMethod:
AuthMethod): ChargingSession
    + stopCharging(): ChargingSession
    + updateSessionProgress(currentSOC: Percentage, powerDelivered: PowerOutput)
    + isAvailable(): boolean
    + isHealthy(): boolean
    + predictMaintenanceNeeded(): boolean  # Nice to have
    + getChargingRate(): PowerOutput
    + markOffline(reason: String)
    + completeSession(): ChargingSession
```

Entity: ChargingSession

```
class ChargingSession (Entity):
    # Identity
    - sessionId: SessionId
    - station: ChargingStation
    - vehicle: ElectricVehicle

    # Authorization
    - authMethod: AuthMethod  # APP, RFID
    - authorizedBy: CustomerId

    # Timing
    - startTime: DateTime
    - endTime: DateTime?
    - chargingDuration: Duration
    - idleDuration: Duration?  # After charge complete
```

```
    # Battery
    - startSOC: Percentage  # State of Charge at start
    - currentSOC: Percentage
    - targetSOC: Percentage?  # User-defined target (e.g., 80%)
    - finalSOC: Percentage?

    # Energy
    - energyConsumed: Energy  # kWh
    - peakPowerDelivered: PowerOutput
    - averagePowerDelivered: PowerOutput

    # Status & Errors
    - status: SessionStatus  # IN_PROGRESS, TARGET_REACHED, COMPLETED,
INTERRUPTED, FAILED
    - errorCode: String?
    - errorMessage: String?

    # Billing
    - chargingFee: Money
    - idleFee: Money?

    Methods:
    + updateProgress(currentSOC: Percentage, powerDelivered: PowerOutput)
    + hasReachedTarget(): boolean
    + complete(): ChargingSession
    + interrupt(reason: String): ChargingSession
    + startIdlePeriod()  # When charging complete but vehicle remains
    + calculateChargingFee(): Money
    + calculateIdleFee(): Money
    + getTotalFee(): Money
    + isInGracePeriod(): boolean  # 15 minutes after charge complete
```

Domain Service: ChargingPricingService

```
class ChargingPricingService (DomainService):
    Methods:
    + calculateChargingFee(
        energyConsumed: Energy,
        powerLevel: PowerLevel,
        membershipTier: MembershipTier
      ): Money
      # Level 2: $0.35/kWh
      # DC Fast: $0.50/kWh
      # Ultra-Fast: $0.50/kWh

    + calculateIdleFee(
        idleDuration: Duration,
        powerLevel: PowerLevel
      ): Money
      # Per-minute fee after 15-min grace
```

Enumerations:

```
enum ConnectorType:
    TYPE2
    CCS
    CHAdeMO

enum PowerLevel:
    LEVEL2        # 7-19 kW
    DC_FAST       # 50-150 kW
    ULTRA_FAST    # 150-350 kW

enum AuthMethod:
    MOBILE_APP
    RFID_CARD

enum HealthStatus:
    HEALTHY
    DEGRADED      # Working but suboptimal
    FAULTY        # Requires immediate attention

enum SessionStatus:
    IN_PROGRESS
    TARGET_REACHED    # User's target SOC achieved
    COMPLETED         # Session ended normally
    INTERRUPTED       # Session stopped prematurely
    FAILED           # Technical error
```

4.3 Customer Management Context

Aggregate: Customer

```
class Customer (AggregateRoot):
    # Identity
    - customerId: CustomerId
    - accountType: AccountType  # INDIVIDUAL, CORPORATE, FLEET, GUEST

    # Personal Info
    - name: PersonName
    - email: Email
    - phone: PhoneNumber

    # Account
    - vehicles: List<CustomerVehicle>
    - membership: Membership
    - preferences: CustomerPreferences
    - paymentMethods: List<PaymentMethod>
    - defaultPaymentMethod: PaymentMethod

    # Corporate-specific
```

```
    - corporateId: CorporateId?  # If corporate account
    - department: String?
    - costCenter: String?
    - approvalLimit: Money?

    # Guest-specific
    - isGuest: boolean
    - guestSessionId: GuestSessionId?

    Methods:
    + registerVehicle(vehicle: Vehicle): CustomerVehicle
    + removeVehicle(vehicleId: VehicleId)
    + upgradeMembership(tier: MembershipTier)
    + addPaymentMethod(method: PaymentMethod)
    + setDefaultPaymentMethod(methodId: PaymentMethodId)
    + getDepartmentalUsage(period: Period): UsageReport  # Corporate
    + getFleetUsage(period: Period): FleetUsageReport  # Fleet
```

Entity: CorporateAccount

```
class CorporateAccount (Entity):
    - corporateId: CorporateId
    - companyName: String
    - taxId: String
    - billingAddress: Address
    - users: List<CustomerId>
    - vehicles: List<VehicleId>
    - departments: List<Department>
    - monthlyBillingCycle: boolean
    - creditLimit: Money
    - currentBalance: Money

    Methods:
    + addUser(customerId: CustomerId, department: Department)
    + addVehicle(vehicleId: VehicleId, assignedTo: CustomerId)
    + getDepartmentalReport(department: Department, period: Period):
DepartmentalReport
    + getMonthlyInvoice(month: YearMonth): CorporateInvoice
```

Enumerations:

```
enum AccountType:
    INDIVIDUAL
    CORPORATE
    FLEET
    GUEST

enum MembershipTier:
```

```
    GUEST      # Pay-per-use, no membership
    BASIC      # Free tier
    SILVER     # Mid-tier benefits
    GOLD       # Premium tier
    PLATINUM   # Top tier
    CORPORATE  # Corporate accounts
```

4.4 Billing Context

Domain Service: ParkingFeeCalculator

```
class ParkingFeeCalculator (DomainService):
    Methods:
    + calculateFee(
        session: ParkingSession,
        slotType: SlotType,
        facility: FacilityId,
        membershipTier: MembershipTier
      ): Money

      # Base rates:
      # Regular: $3/hr
      # EV: $4/hr
      # Handicap: $1.50/hr
      # Compact: $2/hr

      # Apply peak hour multiplier if applicable
      # Apply membership discount
      # Apply daily/weekly/monthly caps

    + applyDailyCap(totalFees: Money): Money  # $25 cap
    + applyWeeklyCap(totalFees: Money): Money  # $120 cap
    + applyMonthlyCap(totalFees: Money): Money  # $300 cap

    + calculateLostTicketFee(facility: FacilityId): Money
    + calculateOvernightRate(facility: FacilityId, duration: Duration): Money
```

Entity: Invoice

```
class Invoice (Entity):
    - invoiceId: InvoiceId
    - customerId: CustomerId
    - accountType: AccountType

    # Parking
    - parkingSessions: List<ParkingSession>
    - parkingSubtotal: Money
```

```
    # Charging (Separate)
    - chargingSessions: List<ChargingSession>
    - chargingSubtotal: Money

    # Totals
    - subtotal: Money
    - tax: Money
    - totalAmount: Money

    # Payment
    - issueDate: DateTime
    - dueDate: DateTime
    - status: InvoiceStatus
    - paymentMethod: PaymentMethod?
    - paidDate: DateTime?

    # Corporate
    - department: String?
    - costCenter: String?

    # Data Retention
    - retentionEndDate: DateTime  # Issue date + 5 years

    Methods:
    + addParkingSession(session: ParkingSession)
    + addChargingSession(session: ChargingSession)
    + calculateTotal(): Money
    + applyMembershipDiscount(tier: MembershipTier): Money
    + markPaid(method: PaymentMethod): Invoice
    + markFailed(): Invoice  # Triggers exit hold for members
    + canBeDeleted(currentDate: DateTime): boolean  # After 5 years
```

---

5. INTEGRATION PATTERNS

5.1 External System Integrations: What do we need to integrate with?

```
Required Integrations:
├── Google Maps API
│   └── Real-time availability overlay
├── Apple Maps API
│   └── Real-time availability overlay
├── Existing Payment Gateway
│   └── Card processing, Apple/Google Pay, PayPal
├── Existing CRM
│   └── Customer data synchronization
└── Gate Hardware
    ├── LPR (License Plate Recognition) cameras
    ├── Barrier gates
    └── Mobile QR code readers
```

5.2 Domain Events (Published Language): What business events exist in our domain?

From Parking Operations Context:

```
- ReservationCreated
- ReservationConfirmed
- ReservationCancelled
- VehicleParked
- VehicleLeft
- GracePeriodStarted
- GracePeriodExpired
- MaxDurationExceeded
- OccupancyChanged (for Maps API)
```

From EV Charging Context:

```
- ChargingSessionStarted
- ChargingProgressUpdated
- TargetSOCReached
- ChargingCompleted
- IdlePeriodStarted (after 15-min grace)
- IdleFeeIncurred
- SessionInterrupted
- StationOffline
- StationMaintenanceRequired
```

From Customer Context:

```
- CustomerRegistered
- VehicleRegistered
- MembershipUpgraded
- CorporateAccountCreated
- DepartmentalReportGenerated
```

From Billing Context:

```
- InvoiceGenerated
- PaymentReceived
- PaymentFailed (triggers exit hold)
- MonthlyCapReached
- CorporateInvoiceSent
```